

Báo cáo Kỹ thuật Dự án: Tối ưu hóa và Triển khai Kiến trúc WHAM trên Nền tảng iOS (Phần cứng iPhone 11 Pro Max)

Báo cáo này trình bày quá trình chuyển đổi và triển khai hệ thống ước lượng tư thế 3D WHAM (World-grounded Humans with Accurate Motion) lên nền tảng iOS. Trọng tâm của báo cáo tập trung vào ba thách thức cốt lõi: mở xẻ cấu trúc đường ống (pipeline) nguyên bản, ứng dụng kỹ thuật Chưng cất Tri thức (Knowledge Distillation) cho mô hình trích xuất đặc trưng hình ảnh, và giải pháp can thiệp mã nguồn để đưa mạng nơ-ron hồi quy (RNN) lên CoreML thông qua cơ chế suy luận bảo toàn trạng thái (Stateful Inference) nhằm khắc phục triệt để hiện tượng tràn bộ nhớ (Out-of-Memory - OOM).

1 Giải phẫu Kiến trúc WHAM và Chiến lược Chuyển đổi

Trong kiến trúc nguyên thủy, WHAM hoạt động như một đường ống phức tạp, kết hợp chặt chẽ giữa thông tin không gian (spatial) và thời gian (temporal). Để khả thi hóa việc triển khai trên iOS với cấu trúc 17 khớp (định dạng COCO), hệ thống đã được bóc tách và tái định hình thành các module độc lập, tối ưu hóa cho việc biên dịch sang định dạng .mlpackage:

- **Mạng trích xuất điểm khớp 2D (2D Keypoint Detector):** WHAM yêu cầu chuỗi tọa độ 2D làm dữ liệu nền tảng. Dự án sử dụng mô hình YOLOv8-pose phiên bản nano (`yolov8n-pose.mlpackage`) để trích xuất 17 khớp COCO trên từng khung hình ảnh độc lập.
- **Mạng tích hợp đặc trưng (Feature Integrator):** Khối này chịu trách nhiệm dung hợp chuỗi điểm 2D với đặc trưng hình ảnh dày đặc (dense features) từ một bộ mã hóa hình ảnh (Vision Transformer). Đây là thành phần tính toán nặng nhất, bắt buộc phải áp dụng kỹ thuật giảm thiểu tham số.
- **Mạng mã hóa/giải mã động học (Motion Encoder & Decoder):** Một mạng RNN đơn hướng tiếp nhận véc-tơ đặc trưng đã tích hợp để liên tục dự đoán tọa độ 3D và quỹ đạo toàn cục. Việc chuyển đổi module này sang iOS gặp rào cản nghiêm trọng về quản lý RAM, đòi hỏi tái cấu trúc ở cấp độ đồ thị tính toán.

2 Tối ưu hóa Đặc trưng Không gian: Chưng cất ViT sang FastViT

Kiến trúc WHAM gốc sử dụng mạng Vision Transformer cỡ lớn (ViT-H/16 từ mô hình HMR2.0) để trích xuất ngữ cảnh hình ảnh. Khối lượng tham số khổng lồ này vượt quá giới hạn tài nguyên của chip A13 Bionic (thiết bị iPhone 11 Pro Max chỉ sở hữu 4GB RAM dùng chung).

Giải pháp được áp dụng là xây dựng một quy trình Chưng cất Tri thức (Knowledge Distillation) trong tệp `utils/distillvit-2.ipynb`:

- **Mô hình Giáo viên (Teacher Model):** Sử dụng kiến trúc HMR2.0 gốc (`hmr2a.ckpt`) với trạng thái đóng băng trọng số hoàn toàn (`requires_grad = False`).
- **Mô hình Học sinh (Student Model):** Khởi tạo lớp `FastViTStudent` dựa trên kiến trúc `fastvit_sa24` từ thư viện `timm`.
- **Dữ liệu:** Sử dụng bộ dữ liệu `COCO Person Only Dataset` trên Kaggle, do các bộ dữ liệu khác được sử dụng bởi nhóm tác giả đều cần xin phép, nên chỉ có thể thay thế bằng bộ dữ liệu này, vì tính chất tương đương. Tất nhiên vì độ đa dạng dữ liệu không bằng bản gốc, nên mô hình distill dù có loss chỉ 0.0134 ở epoch cuối, nhưng không thể đảm bảo bias đủ nhỏ so với mô hình gốc.

Cơ sở lựa chọn FastViT

Kiến trúc FastViT mang lại hiệu năng suy luận đột phá trên thiết bị di động nhờ kỹ thuật Tái tham số hóa cấu trúc. Cụ thể, khối `RepMixer` loại bỏ các kết nối tắt trong quá trình suy luận, giúp giảm thiểu chi phí truy xuất bộ nhớ. Đồng thời, việc thay thế các lớp self-attention ở các tầng sơ cấp bằng large kernel convolutions giúp mở rộng receptive field mà không làm suy giảm tốc độ.

Đồng bộ hóa Không gian Biểu diễn

Để tương thích với đường ống WHAM, mô hình học sinh được bổ sung một lớp kết nối đầy đủ (`nn.Linear`) tại khối đầu ra. Lớp này làm nhiệm vụ project đặc trưng của FastViT vào không gian $\mathbb{R}^{1024}$, khớp với số chiều đầu vào của mạng RNN phía sau.

3 Phá vỡ Điểm nghẽn Cốt lõi: Suy luận Bảo toàn Trạng thái cho RNN

Thách thức kỹ thuật phức tạp nhất của dự án nằm ở quá trình chuyển đổi khối Motion Encoder/Decoder.

Nguyên nhân gây tràn bộ nhớ (OOM Crash)

Bản chất của WHAM là dự đoán đệ quy theo thời gian thông qua RNN. Khi biên dịch mô hình này sang CoreML, trình biên dịch của Apple xử lý các vòng lặp đồ thị động bằng cách “trải phẳng” (unroll) toàn bộ chuỗi theo thời gian. Đối với một chuỗi đầu vào có độ dài N khung hình, CoreML sẽ nhân bản kiến trúc RNN thành N lớp Feed-Forward tĩnh trong bộ nhớ. Sự phình to của đồ thị tính toán tĩnh này ngay lập tức vắt kiệt RAM khả dụng, dẫn đến việc hệ điều hành buộc kill ứng dụng ngay khi tải mô hình.

Các thử nghiệm thất bại

Do bản chất mạng RNN với LSTM gốc của mạng WHAM đã rất nhẹ, việc chưng cất nó sang một mô hình nhỏ hơn nữa có thể không đảm bảo độ chính xác chấp nhận được. Giải pháp đầu tiên được thử nghiệm là chuyển trọng số mô hình sang FP16, giúp kích thước mô hình còn 123MB. Tuy nhiên, nhóm tác giả thiết lập mô hình xử lý 81 frame liên tiếp nhau, nên khi duỗi ra, kích thước này lên tới $123 \times 81 \text{MB}$ và xảy ra tràn RAM. Khi chuyển mô hình sang dạng INT8, kích thước chỉ còn 60MB, nhưng vẫn quá lớn ($60 \times 81 = 4860 \text{MB}$). Thậm chí, khi áp dụng kỹ thuật Palettization (một kỹ thuật lượng tử hoá của Apple), giảm kích thước model xuống dạng 6 bit, thì vẫn quá lớn so với bộ nhớ. Lúc này, chìa khóa chỉ có thể nằm ở việc ngăn chặn hành vi tự duỗi mạng của CoreML.

Giải pháp: Tách biệt đồ thị tính toán (Init và Step)

Để ngăn chặn cơ chế unroll của CoreML, đồ thị RNN nguyên khối được can thiệp ở mức mã nguồn và chia tách thành hai mô hình độc lập:

1. Giai đoạn Khởi tạo (Init Model):

- Chỉ thực thi một lần duy nhất tại khung hình đầu tiên.
- Tiếp nhận tọa độ 2D và đặc trưng $\mathbb{R}^{1024}$ để khởi tạo trạng thái ẩn ban đầu h_0 .

2. Giai đoạn Suy luận Liên tục (Step Model):

- Xử lý tuần tự từng khung hình theo thời gian thực (Sequence Length = 1).
- Khối Step nhận hai đầu vào: Đặc trưng khung hình hiện tại x_t và Trạng thái ẩn từ khung hình trước đó h_{t-1} .
- Mô hình thực hiện một bước tiến (forward pass) duy nhất để xuất ra tọa độ 3D y_t và cập nhật trạng thái ẩn mới h_t .

Trạng thái h_t được lưu trữ trực tiếp trong bộ nhớ ứng dụng thông qua Swift và truyền ngược lại làm đầu vào cho khung hình $t + 1$. Bằng cách này, đồ thị tính toán của CoreML bị ép xuống kích thước tối thiểu (chỉ bằng một khung hình), trong khi hệ thống vẫn duy trì được toàn bộ tính liên kết thời gian (temporal consistency). Giải pháp này giải quyết triệt để lỗi OOM, giúp mô hình chạy trơn tru trên chip A13 Bionic.

4 Tiền đề Kỹ thuật cho Nâng cấp SMPL-24

Việc áp dụng phương pháp Init/Step kết hợp cùng FastViT đã giải phóng một lượng lớn tài nguyên CPU, GPU và Apple Neural Engine (ANE). Hệ thống hiện vận hành ổn định ở định dạng COCO-17 với tốc độ tương đối nhanh.

Khoảng trống tài nguyên này là cơ sở vững chắc để khẳng định tính khả thi của việc nâng cấp lên cấu trúc 24 khớp (SMPL-24). Khác biệt lớn nhất giữa định dạng 17 khớp và 24 khớp nằm ở quá trình giải quyết bài toán Động học Thuận (Forward Kinematics), đòi hỏi tính toán hàng loạt các chuỗi nhân ma trận xoay cục bộ. Với việc chip A13 Bionic được trang bị bộ đồng xử lý ma trận AMX (Apple Matrix Coprocessor), việc tích hợp thuật toán Forward Kinematics vào đường ống Step (thông qua `MLCustomLayer` trong Swift) là hoàn toàn khả thi và bảo đảm được hiệu năng.